

React Interview Prep

useCallback & useMemo — Complete Guide

Reference Equality · The Problem · useMemo · useCallback · Deps · When NOT to Use · Q&A;

- | | |
|----------|--|
| ■ Covers | All examples from conversation — nothing skipped |
| ■ Series | React Interview Prep — useCallback & useMemo Deep Dive |

by Akshay Chavhan • React Interview Series

1. Why These Hooks Exist — The Root Problem

Every re-render, React runs the component function top to bottom — recreating every variable, object, array and function from scratch.

```
function App() {  
  const user = { name: 'Akshay' }; // new object every render  
  const nums = [1, 2, 3]; // new array every render  
  const onClick = () => {}; // new function every render  
}
```

■ **Problem: JS compares objects/arrays/functions by REFERENCE. `{}` `===` `{}` is false. New ref every render = child always sees 'changed' prop = unnecessary re-render.**

Reference Equality — The Root Cause

```
1 === 1 // true ■ primitives compared by VALUE  
'hi' === 'hi' // true ■  
{ } === { } // false ■ objects compared by REFERENCE  
[] === [] // false ■  
() => { } === () => { } // false ■  
  
const a = { name: 'Akshay' };  
const b = { name: 'Akshay' };  
a === b // false ■ – same value, different memory location  
const c = a; a === c // true ■ – same reference
```

2. useMemo — Memoize a VALUE

useMemo runs a function, caches the RESULT, and returns it. React only re-runs when dependencies change.

```
const result = useMemo(() => {  
  return expensiveComputation(a, b);  
}, [a, b]); // re-run only when a or b changes
```

Use Case 1 — Heavy Computation

```
// Expensive – avoid recalculating on unrelated re-renders  
const sorted = useMemo(() => {  
  console.log('sorting...');  
  return data.sort((a,b) => b.score - a.score);  
}, [data]); // only re-sort when data changes ■
```

Use Case 2 — Stable Object/Array Reference

```
// NOT heavy – but new ref every render breaks React.memo on child
```

■ Wrong

```
// New object every render  
function App() {  
  const user = { name: 'Akshay' };  
  // new reference every render!  
  return <Child user={user} />;  
  // Child re-renders every time ■
```

■ Correct

```
// Stable reference with useMemo  
function App() {  
  const user = useMemo(() => ({  
    name: 'Akshay'  
  }), []); // same ref if no deps  
  change  
  return <Child user={user} />  
  // Child skips re-render ■
```

■ *useMemo is NOT only for heavy functions. Also use to stabilize object/array refs passed to children.*

When Does useMemo Execute?

MOUNT → runs immediately, result cached ■
RE-RENDER (dep unchanged) → skips, returns cache ■
RE-RENDER (dep changed) → re-runs, new cache
BUTTON CLICK (dep unchanged) → does NOT re-run ■

■ *REACT controls when useMemo runs. YOU cannot call useMemo manually. It returns a VALUE, not a function.*

3. useCallback — Memoize a FUNCTION

useCallback stores the function itself and returns the SAME reference across renders unless dependencies change. YOU call it when needed.

```
const handleDelete = useCallback((id) => {  
  setUsers(prev => prev.filter(u => u.id !== id));  
}, []); // same function reference across renders ■
```

MOUNT → function stored in memory ■ (NOT called)

RE-RENDER → same ref returned (deps unchanged) ■

BUTTON CLICK → function body EXECUTES ■ (YOU control this)

Click 100x → executes 100 times ■ (deps never block calls)

■ **YOU control when useCallback's function runs (by calling it). Deps only control memory recreation. useCallback NEVER stops the function from executing when called.**

useCallback Alone is NOT Enough — Must Pair with React.memo

■ Wrong

```
// Child not memoized  
// re-renders anyway – useCallback  
wasted!  
  
const Child = ({ onClick }) => (  
  <button onClick={onClick}/>  
);  
// useCallback = pointless here ■
```

■ Correct

```
// Child wrapped in React.memo ■  
  
const Child = React.memo(  
  ({ onClick }) => (  
    <button onClick={onClick}/>  
  )  
);  
// useCallback + memo = works ■
```

■ **useCallback + React.memo = the pair. Without React.memo, useCallback does nothing useful.**

4. [] vs [dep] — What Changes

Deps	useCallback recreates?	useMemo re-runs?	When?
[] empty	Never after mount	Never after mount	Only on first mount
[dep]	When dep changes	When dep changes	Mount + on dep change
No array	Every render	Every render	Avoid — pointless

Stale Closure Bug — Why Deps Matter

■ Wrong

```
// count not in deps – STALE
const fn = useCallback(() => {
  console.log(count); // always 0!
}, []); // count missing ■
// captured count=0 at mount
// never updated
```

■ Correct

```
// count in deps – FRESH
const fn = useCallback(() => {
  console.log(count); // latest ■
}, [count]);
// recreated when count changes
```

■ **Rule:** If your function **READS** a variable from outside → add it to deps. State setters (`setUsers`, `setCount`) are always stable — never need to be in deps.

5. Does useCallback Prevent Function Execution? (Common Misconception)

NO. `useCallback` controls MEMORY (recreation). It NEVER controls EXECUTION (running). Clicking a button 100 times with same deps = function executes 100 times.

What you do	useCallback effect	Function executes?
Click button (same value)	No recreation ■	Yes, always ■
Click button (different value)	Depends on deps	Yes, always ■
Re-render with same deps	No recreation ■	Not called (nobody clicked)
Re-render with changed deps	Recreates function	Not called (nobody clicked)

■ **useCallback prevents the function from being BORN AGAIN in memory. It never prevents the function from DOING ITS JOB when called.**

6. useMemo — REACT Controls It. You Cannot Call It.

Click button 100 times, deps never change → `useMemo` executes ZERO times. Returns cached result every time.

	useCallback	useMemo
Returns	A function	A value (number, obj, array)

Who triggers	YOU (by calling it)	REACT (on render)
Same dep + button click	Executes every click ■	Does NOT re-run ■
Same dep + re-render	Not recreated ■	Not re-run ■
Can call manually	Yes ■	No ■
Use for	Stable function ref	Expensive calc + stable objects

// They are equivalent – different convenience API:

```
useCallback(fn, deps) === useMemo(() => fn, deps)
```

7. When to Use — Decision Guide

Situation	Use?	Which?
Passed to React.memo child	■ Yes	useCallback
In useEffect deps array	■ Yes	useCallback
Expensive computation on render	■ Yes	useMemo
Object/array passed to child	■ Yes	useMemo
Simple inline button handler	■ No	Neither
Child has no React.memo	■ No	Neither
Just to be safe / everywhere	■ No	Neither — adds cost

■ **Both hooks have memory + comparison cost. Only use when you can clearly see or measure the re-render problem. Premature optimization = complexity for nothing.**

8. Interview Q&A;

Q

Beginner

Q1. What is useCallback?

→ useCallback returns a memoized function — same reference across renders unless deps change. It prevents unnecessary child re-renders when functions are passed as props to React.memo components.

Q

Beginner

Q2. What is useMemo?

→ useMemo runs a function, caches the result, and returns it. React only re-runs it when dependencies change. Used for expensive computations and to stabilize object/array references passed to children.

Q

Intermediate

Q3. Does useCallback prevent the function from executing?

→ No — useCallback only controls memory recreation. The function still executes every time you call it. Clicking a button 100 times with same deps will execute the function 100 times. Deps only decide whether a new function is created in memory.

Q

Intermediate

Q4. What is the difference between [] and [dep] in useCallback?

→ [] means the function is never recreated after mount — same reference forever. [dep] means the function is recreated whenever dep changes. Use [dep] when the function reads that variable inside it, otherwise you get a stale closure bug.

Q**Intermediate****Q5. Is useMemo only for heavy/expensive functions?**

→ No. useMemo has two use cases: expensive computations to avoid recalculating, AND stabilizing object/array references so child components don't re-render due to new reference every render. Both are equally important.

Q**Intermediate****Q6. Can you call useMemo like a function?**

→ No. useMemo returns the computed VALUE directly — not a function. You cannot call it manually. React controls when it runs — on mount and when deps change. useCallback is different — it gives you a function YOU call.

Q**Expert****Q7. Does useCallback alone prevent child re-renders?**

→ No. useCallback must be paired with React.memo on the child component. useCallback ensures the function reference stays stable. React.memo ensures the child skips re-rendering when props haven't changed. Without both, neither works.

Q**Expert****Q8. When should you NOT use useCallback or useMemo?**

→ When the child isn't wrapped in React.memo, when the computation is trivial, when the function is only used inline in the same component, or when adding it just to be safe. Both hooks have memory and comparison overhead — overusing them adds complexity with no benefit.